

OpenWRT NetBird

version 32

Latest version:

<https://raw.githubusercontent.com/egc112/OpenWRT-egc-add-on/main/notes/OpenWRT%20Netbird.pdf>

This is a W.I.P., comments are welcome.

Introduction

NetBird is a modern, fully open-source Zero Trust VPN that provides a simple and secure way to connect users, devices, and services. It is built on WireGuard®, offering fast, encrypted peer-to-peer networking without the complexity of traditional VPN setups.

Unlike some alternatives, NetBird is fully open source and allows the entire control plane to be self-hosted. This can give full control over your networking infrastructure and data, and avoids dependency.

You can use it for remote access to your home network, to connect multiple routers and other clients (phone/PC/Mac etc.) and when setup as exit node also as a remote VPN.

Usually you can do the same by setting up your own WireGuard server and clients.

[WireGuard Server Setup Guide](#)

[WireGuard Client Setup Guide](#)

But this only works if you have at least a public IP address on one side of the connection.

If you are behind CGNAT, so do not have a public IPv4 address and also do not have a public IPv6 address (check with: *ifstatus wan6*) or using IPv6 is not applicable then you have to involve a third party as man-in-the-middle.

This can be a VPN provider which supports port forwarding (e.g. ProtonVPN), or you can rent a Virtual Private Server (I have an Oracle VPS which can be had for free, see at the bottom of this guide), or use things like [NetBird](#), [Zerotier](#) (also layer 2), [Cloudflared](#), [Tailscale](#) or [ngrok](#) and there are more.

I favor NetBird because it is fully open source and has some [advantages](#) over Tailscale, but it still is a commercial third party however a free tier is available which is sufficient for small to medium sized networks with up to 5 users (admins) and 100 peers (routers, VPS, client PC, phone, etc.).

Big advantage of NetBird is that it is fully open source and you can self host the control plane (Dashboard) in which case you are your own third party, self hosting the control plane is outside the scope of this paper.

Start with viewing: <https://docs.netbird.io/how-to/getting-started>

All the docs can be found at: <https://docs.netbird.io/>

A [NetBird wiki](#) is in the works

A very useful OpenWRT thread for discussion and support: <https://forum.openwrt.org/t/netbird-support-discussion-thread/237831>

Luci-app-netbird

A new luci-app for Netbird is in the works, see:

<https://forum.openwrt.org/t/luci-app-netbird-web-ui-for-netbird-on-openwrt/251349>

Table of Contents

Introduction.....	1
Luci-app-netbird.....	1
Make a free account on NetBird.....	3
MFA (Multi Factor Authentication).....	3
Install NetBird on OpenWRT router.....	5
Setup on Router with SSO login.....	6
Setup on router with manual made key.....	7
Create a setup key for your OpenWRT router: NetBird Dashboard > Setup Keys:.....	7
Post installation.....	8
Network setup.....	9
Firewall setup.....	9
Check and Troubleshoot.....	11
Interface.....	11
Routing.....	11
Status.....	11
Info.....	11
WireGuard.....	11
Log.....	11
OpenWRT Forum.....	11
Online.....	11
Allow SSH access from Dashboard.....	12
Changes Starting with version 0.60.....	13
Create Routes.....	14
Create Networks.....	17
DNS settings.....	21
Create Exit node.....	22
NetBird together with other VPN interfaces and Policy Based Routing.....	25
IP rule priority.....	25
Overlapping FWMARK.....	25
Default route of Netbirds WG connection.....	26
IPv6 and Netbird.....	27
IPv6.....	27
Support.....	27
Known Problems.....	27
SSH-Access from Dashboard.....	27
Install on Oracle VPS with Ubuntu (24.04).....	27
Throughput improvements via transport layer offloading.....	28
Setup Oracle free OpenVPN cloud server.....	29
References.....	29
NetBird Releases.....	29
Configuration.....	29
Addendum 1.....	30
Addendum NB variables.....	38

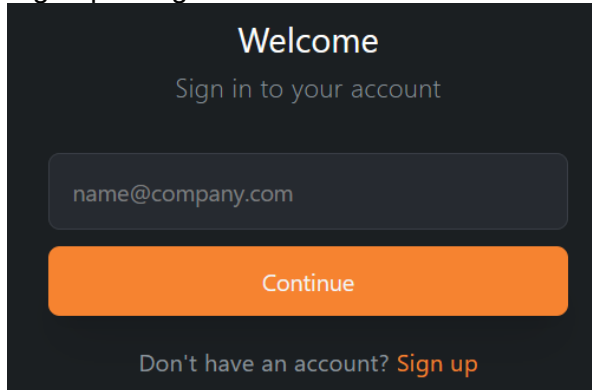
Make a free account on NetBird

go to: <http://netbird.io>

Click:

Get started - free

Sign up or login:

A dark-themed login screen with the title "Welcome" and subtitle "Sign in to your account". It features a text input field containing "name@company.com" and an orange "Continue" button. At the bottom, it says "Don't have an account? Sign up" with "Sign up" in orange.

Welcome

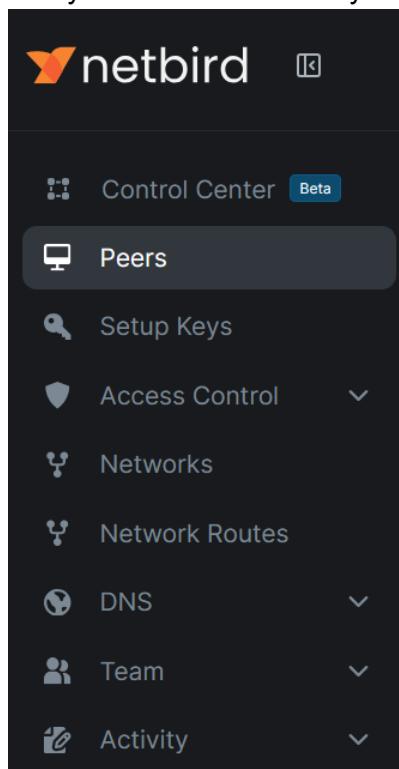
Sign in to your account

name@company.com

Continue

Don't have an account? Sign up

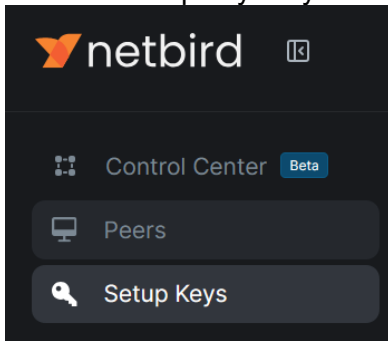
Now you are connected to your NetBird Dashboard the central administration (<https://app.netbird.io>):



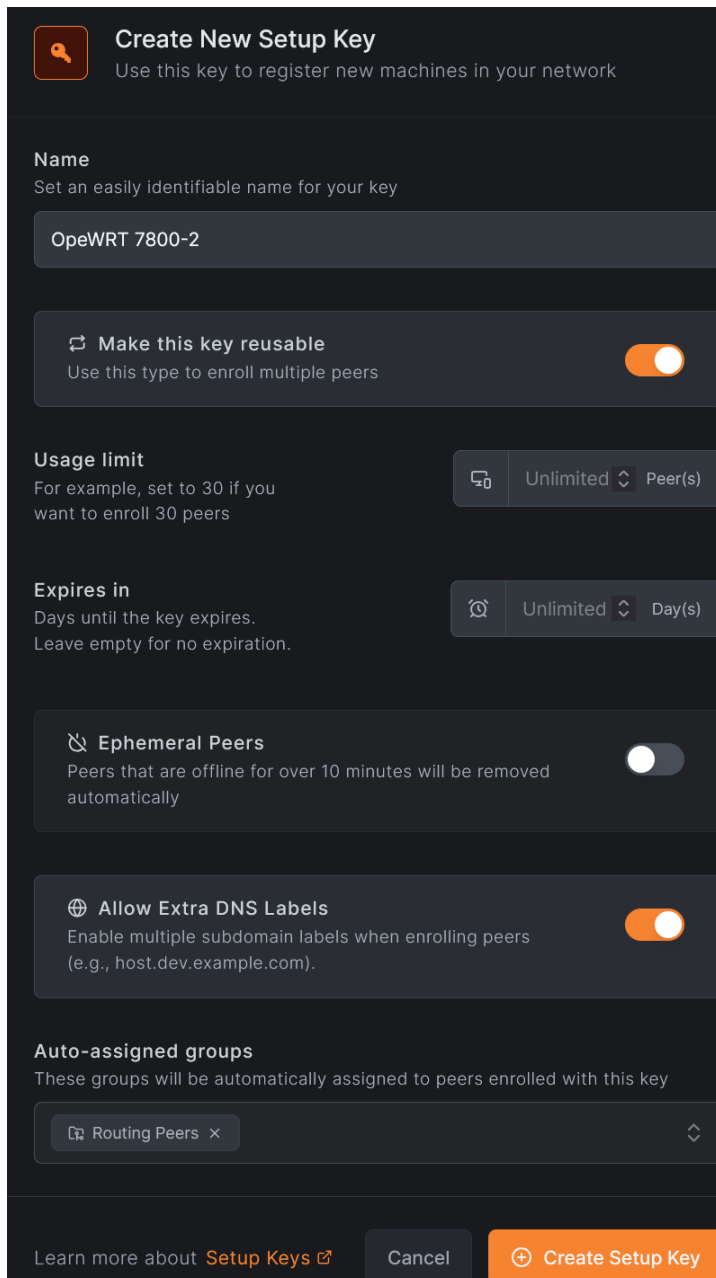
MFA (Multi Factor Authentication)

If you click on your Icon in the upper right hand corner > Profile Settings: you can enable NetBird MFA for added security.

Create a setup key for your OpenWRT router: **NetBird Dashboard > Setup Keys:**



Fill in the name of your router and change the other items, shown are my settings, when done Click *Create Setup Key*.

A screenshot of the 'Create New Setup Key' form in the NetBird dashboard. The form is titled 'Create New Setup Key' and includes a subtitle 'Use this key to register new machines in your network'. It contains several sections: 'Name' with a text input field containing 'OpeWRT 7800-2'; 'Make this key reusable' with a toggle switch turned on; 'Usage limit' with a dropdown menu set to 'Unlimited' and 'Peer(s)'; 'Expires in' with a dropdown menu set to 'Unlimited' and 'Day(s)'; 'Ephemeral Peers' with a toggle switch turned off; 'Allow Extra DNS Labels' with a toggle switch turned on; and 'Auto-assigned groups' with a dropdown menu showing 'Routing Peers'. At the bottom, there are three buttons: 'Learn more about Setup Keys', 'Cancel', and 'Create Setup Key'.

Copy and store the setup key

Install NetBird on OpenWRT router

NetBird is a rather large package around 12 MB written in Go so make sure your storage is sufficient, if you install it as a package it will take up double this storage, if your storage is not sufficient then make a new build using e.g. the [firmware selector](#) and *customize installed packages* adding *netbird*.

If you already have a running build then a very handy tool to make a list for the firmware selector is [owut](#) (*owut list*).

You can even use it for upgrading and adding a package like netbird (*owut upgrade --add netbird*).

Installing package

opkg (24.10):

opkg update

opkg install netbird

apk (25.12 and higher):

apk update

apk add netbird

NetBird is added as a service to OpenWRT which can be called from */etc/init.d/netbird*

See **service netbird help** for commands but the regular commands are available

Make the NetBird service start at boot up with **service netbird enable** but this should be done automatically.

Having the service start is imperative for the right config path as that is set in the service profile.

The NetBird executable is stored in */usr/bin/netbird*.

You can use **netbird help** to see the available commands e.g.:

netbird up/down/status etc.

The config file (>0.55) is stored in */root/.config/netbird*, you might add this path to */etc/sysupgrade.conf*, so that it is included in the backup!

Setup on Router with SSO login

After installing NetBird check that the OpenWRT service is running with `service netbird status`, if this is the case you can Run UP command to log in with SSO (interactive login): `netbird up`

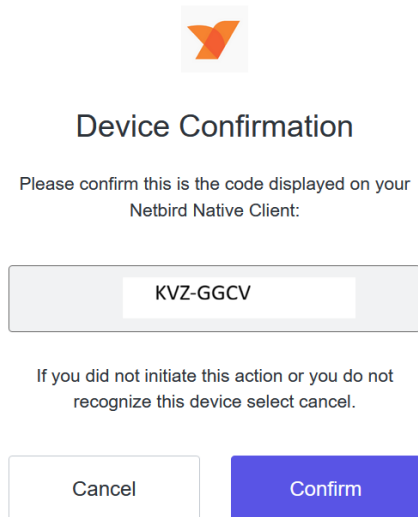
You will see the following text on the console:

Please do the SSO login in your browser.

If your browser didn't open automatically, use this URL to log in:

`https://login.netbird.io/activate?user_code=ZKZX-AACCV`

On your PC where you have opened the NetBird dashboard use your browser to login with the link (user_code) from the routers console and you should see:



Confirm the device and go to your NetBird Dashboard where you should see the new Peer been added

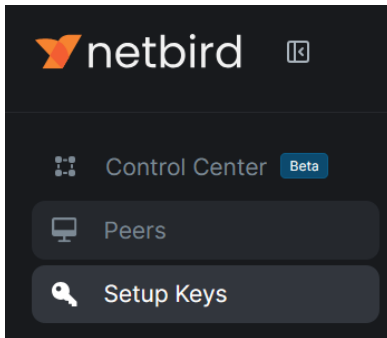
If this is successful proceed to [Post installation](#), [Network setup](#) and [Firewall setup](#)

|

If this does **not** work go to the next section, [Setup on router with key](#) and manually make a peer key on the Dashboard and use that peer key to setup NetBird.

Setup on router with manual made key

Create a setup key for your OpenWRT router: **NetBird Dashboard > Setup Keys:**



Fill in the name of your router and change the other items, shown are my settings, when done Click *Create Setup Key*.



Create New Setup Key

Use this key to register new machines in your network

Name
Set an easily identifiable name for your key

OpeWRT 7800-2



Make this key reusable
Use this type to enroll multiple peers

☒

Usage limit
For example, set to 30 if you want to enroll 30 peers



Unlimited

Peer(s)

Expires in
Days until the key expires.
Leave empty for no expiration.



Unlimited

Day(s)



Ephemeral Peers
Peers that are offline for over 10 minutes will be removed automatically

☐



Allow Extra DNS Labels
Enable multiple subdomain labels when enrolling peers
(e.g., host.dev.example.com).

☒

Auto-assigned groups
These groups will be automatically assigned to peers enrolled with this key



Routing Peers

x

Learn more about [Setup Keys](#)

Cancel

Create Setup Key

Copy and store the setup key

Head back to the console of the router and execute:

```
netbird up --setup-key <key from previous step>
```

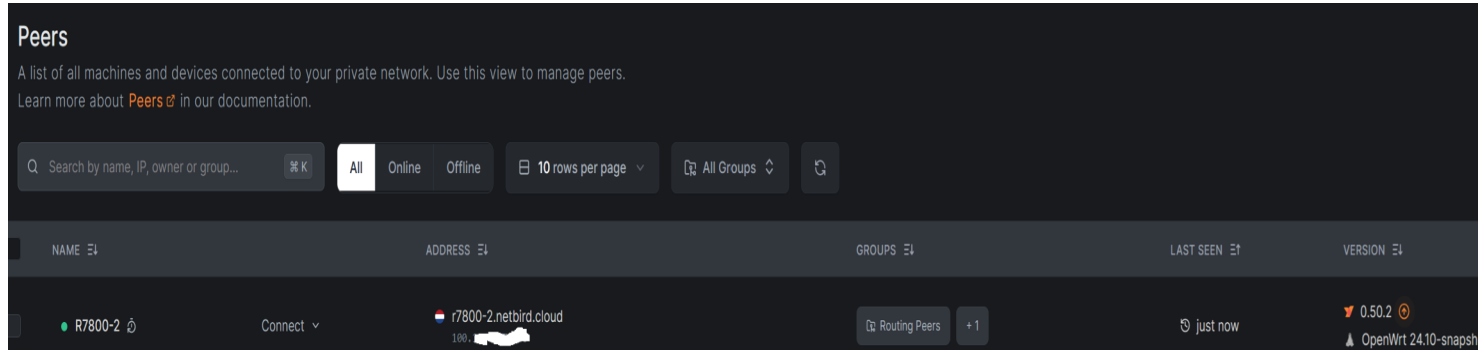
After some time you will see:

```
root@R7800-2:~# netbird up --setup-key E20033F4-0XXXXXXXXXXXXXXXXX
```

Connected

```
root@R7800-2:~#
```

In your Dashboard you can now see the installed peer



with ifconfig or ip address show on the router, you should see the new interface (device) **wt0**

|

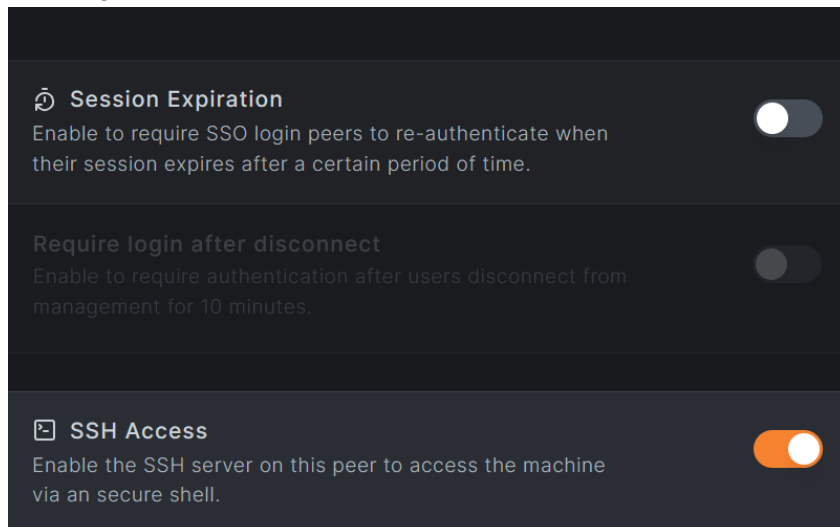
Check NetBird status with: *netbird status* and *service netbird status* both should output: *running*.

If not reboot and check again

Post installation

Open the Peer in the Dashboard by clicking on it in and change the settings.

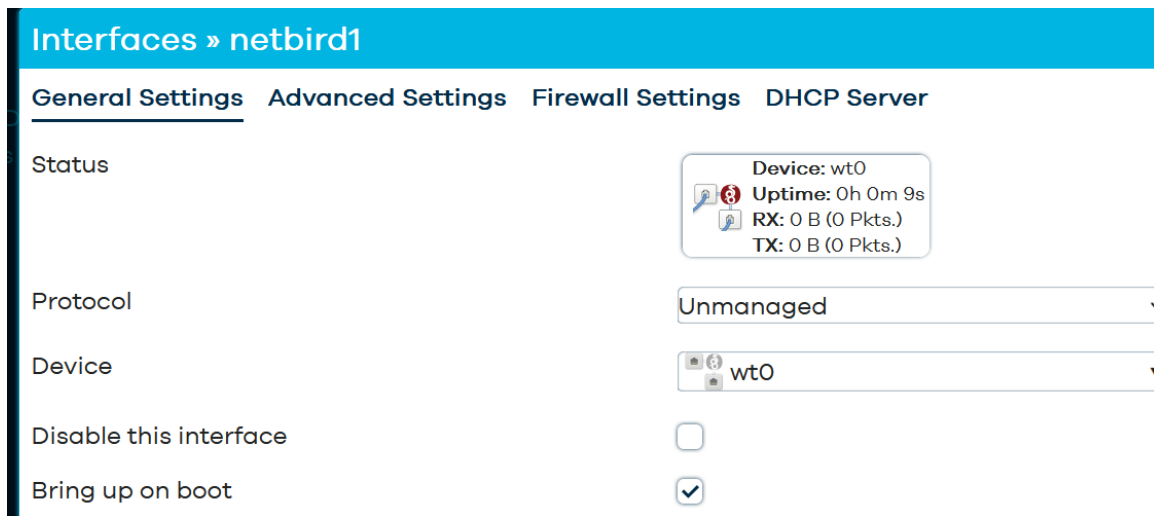
You might want to disable Session Expiration and Enable SSH Access:



Network setup

Create a new unmanaged interface via LuCi: **Network > Interfaces > Add new interface**

- Name: **netbird1**
- Protocol: **Unmanaged**
- Device: **wt0** #For compatibility e.g. with e.g. PBR always name your interface e.g. **wtX**



Interfaces » netbird1

General Settings Advanced Settings Firewall Settings DHCP Server

Status

Device: wt0
Uptime: 0h 0m 9s
RX: 0 B (0 Pkts.)
TX: 0 B (0 Pkts.)

Protocol: Unmanaged

Device: wt0

Disable this interface: ☐

Bring up on boot: ☒

```
/etc/config/network:
config interface 'netbird1'
    option proto 'none'
    option device 'wt0'
```

Firewall setup

Create a new firewall zone via LuCi: **Network → Firewall → Zones → Add**

- Name: **netbird**
- Input: **ACCEPT** (default)
- Output: **ACCEPT** (default)
- Forward: **ACCEPT**
- Masquerading: **on** (might not be necessary)
- MSS Clamping: **on** (might not be necessary when using [0.59.12](#))
- Covered networks: **netbird1**
- Allow forward to destination zones: Select your **LAN** (and/or other internal zones or WAN if you plan on using this device as an exit node), as this is an exit node **WAN** is selected, if you are going to use a VPN and traffic from the exit node should go via this VPN then also add the VPN interface!
- Allow forward from source zones: Select your **LAN** (and/or other internal zones or leave it blank if you do not want to route LAN traffic to other NetBird hosts)

Click **Save & Apply**

Firewall - Zone Settings

General Settings Advanced Settings Contrack Settings

This section defines common properties of "netbird". The *input* and *output* options set the default policies for traffic entering or leaving the zone. The *covered networks* option describes the policy for forwarded traffic between different networks within the zone. *Covered networks* specifies which networks are covered by this zone.

Name	netbird
Input	accept
Output	accept
Intra zone forward	accept
IPv4 Masquerading	☒ Enable network address and port translation IPv4 (NAT4 or NAT6) typically enabled on the wan zone.
MSS clamping	☒
Covered networks	netbird1:

The options below control the forwarding policies between this zone (netbird) and other zones. *Destination zones* cover forwarded traffic from other zones **targeted at netbird**. *Source zones* match forwarded traffic from other zones **targeted at netbird**. The forwarding rule is *unidirectional*, e.g. a forwarding rule to forward from wan to lan as well.

Allow forward to *destination zones*:

lan lan:	wg_stos_6: (empty)	ovpn_client tun1: (empty)	wg_mullvad_se:	wg_proton_nl: (empty)	mullvad_ro: (empty)	wan wan:	wan6:
-----------------	--------------------	----------------------------------	----------------	-----------------------	---------------------	-----------------	-------

Allow forward from *source zones*:

lan lan:	wg_stos_6: (empty)
-----------------	--------------------

/etc/config/firewall:

```
config zone
    option name 'netbird'
    option input 'ACCEPT'
    option output 'ACCEPT'
    option forward 'ACCEPT'
    option masq '1'
    option mtu_fix '1'
    list network 'netbird1'
```

```
config forwarding
    option src 'netbird'
    option dest 'lan'
```

```
config forwarding
    option src 'lan'
    option dest 'netbird'
```

As this is an exit node traffic from netbird to wan is allowed

```
config forwarding
    option src 'netbird'
    option dest 'wan'
```

In the end **reboot** the router or do *service network restart*, *service firewall restart* and *service netbird restart*.

Check and Troubleshoot

Interface

`ip address show wt0`

```
31: wt0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1280 qdisc noqueue state UNKNOWN group
default qlen 1000
    link/none
    inet 100.105.224.116/16 brd 100.105.255.255 scope global wt0
        valid_lft forever preferred_lft forever
```

Routing

`ip route`

```
default via 192.168.0.1 dev wan proto static src 192.168.0.9
100.105.0.0/16 dev wt0 proto kernel scope link src 100.105.224.116
```

Status

`netbird status --detail` to get a detailed status report which also will show if you have a fast P2P connection or a relayed connection.

Info

`service netbird info` to get the info from the OpenWRT ubus service e.g. the NB_STATE_DIR, `service netbird status` will show if the service is running. Use `service netbird help` for more commands

WireGuard

`wg show` this shows the connections to peers and the Allowed IPs which should also show the subnet routes if you made any.

Log

`cat /var/log/netbird/client.log`

You can increase the log level by starting NetBird with `--log-level notice|debug` (see: <https://docs.netbird.io/get-started/cli>)

OpenWRT Forum

<https://forum.openwrt.org/t/netbird-support-discussion-thread/237831/1>

Online

See: <https://docs.netbird.io/help/troubleshooting-client>

Allow SSH access from Dashboard

In the **NetBird Dashboard** open the peer and *Enable SSH Access*:

The screenshot shows the 'DL-WRX36' peer settings in the NetBird dashboard. On the left, there's a list of fields: NetBird IP Address (100.105.224.116), Public IP Address (2001:1000:0000:0000:0000:0000:0000:0000), Domain Name (dl-wrx36.netbird.cloud), Hostname (DL-WRX36), Region (The Netherlands), and Operating System (OpenWrt snapshot). On the right, there are two sections: 'Session Expiration' with a toggle switch, and 'SSH Access' with a toggle switch that is currently turned on. Below these is a 'Remote Access' section with a button labeled '> SSH'.

On the router

Make sure SSH is allowed (<https://github.com/netbirdio/netbird/issues/2632>):

netbird down

netbird up --allow-server-ssh

On your NetBird dashboard you should now be able to SSH into your router:

Dashboard > Peers > Connect dropdown and click SSH:

The screenshot shows the 'Peers' section of the NetBird dashboard. It displays a list of 2 peers. The first peer is 'DL-WRX36' with a status of 'Online'. To the right of the peer name is a 'Connect' dropdown menu. A red arrow points to the '> SSH' button that appears in the dropdown menu.

Connect with the default port 44338 to the in NetBird included SSH server:

The screenshot shows the SSH connection dialog in the NetBird dashboard. It has a title bar with a terminal icon and the text 'DL-WRX36 Connect to 100.105.224.116 via SSH'. Below the title bar, there's a section titled 'Username & Port' with the subtitle 'The username and port you will use to connect to the remote host.' There are two input fields: one for the username 'root' and one for the port '44338'. At the bottom, there are three buttons: 'Learn more about SSH', 'Cancel', and 'Connect'.

Changes Starting with version 0.60

<https://docs.netbird.io/manage/peers/ssh>

<https://forum.netbird.io/t/upcoming-breaking-change-to-netbird-ssh/292>

You need to start NetBird with:

```
netbird up --allow-server-ssh --disable-ssh-auth --enable-ssh-root
```

Furthermore you need to add an Access policy for port 22 and port 22022 (when using the Standard Dropbear SSH) in the NetBird Dashboard:

Access Control > Policies > Add Policy:

The screenshot shows the 'Add Policy' form in the NetBird dashboard. At the top, there are tabs for 'Policy' (selected), 'Posture Checks', and 'Name & Description'. The 'Protocol' section has a dropdown menu set to 'TCP'. Below this, there are 'Source' and 'Destination' sections, each with a dropdown menu set to 'All'. The 'Ports' section has a text input field containing '22' and '22022'. At the bottom, there is a toggle switch for 'Enable Policy' which is currently turned on.

Start the SSH with user root and port 22.

If you have Dropbear running (which is the default in OpenWRT) instead of OpenSSH (sshd) then you need to use port 22022 and also make an Access Policy for port 22022

The screenshot shows the 'Username & Port' form in the NetBird dashboard. The 'Username' field is set to 'root' and the 'Port' field is set to '22'.

The screenshot shows the 'Username & Port' form in the NetBird dashboard. The 'Username' field is set to 'root' and the 'Port' field is set to '22022'.

Create Routes

See: <https://docs.netbird.io/how-to/routing-traffic-to-private-networks>

Note for routing between your peers it is imperative that all involved subnets are unique!

My DL-WRX36 has subnet 192.168.9.0/24.

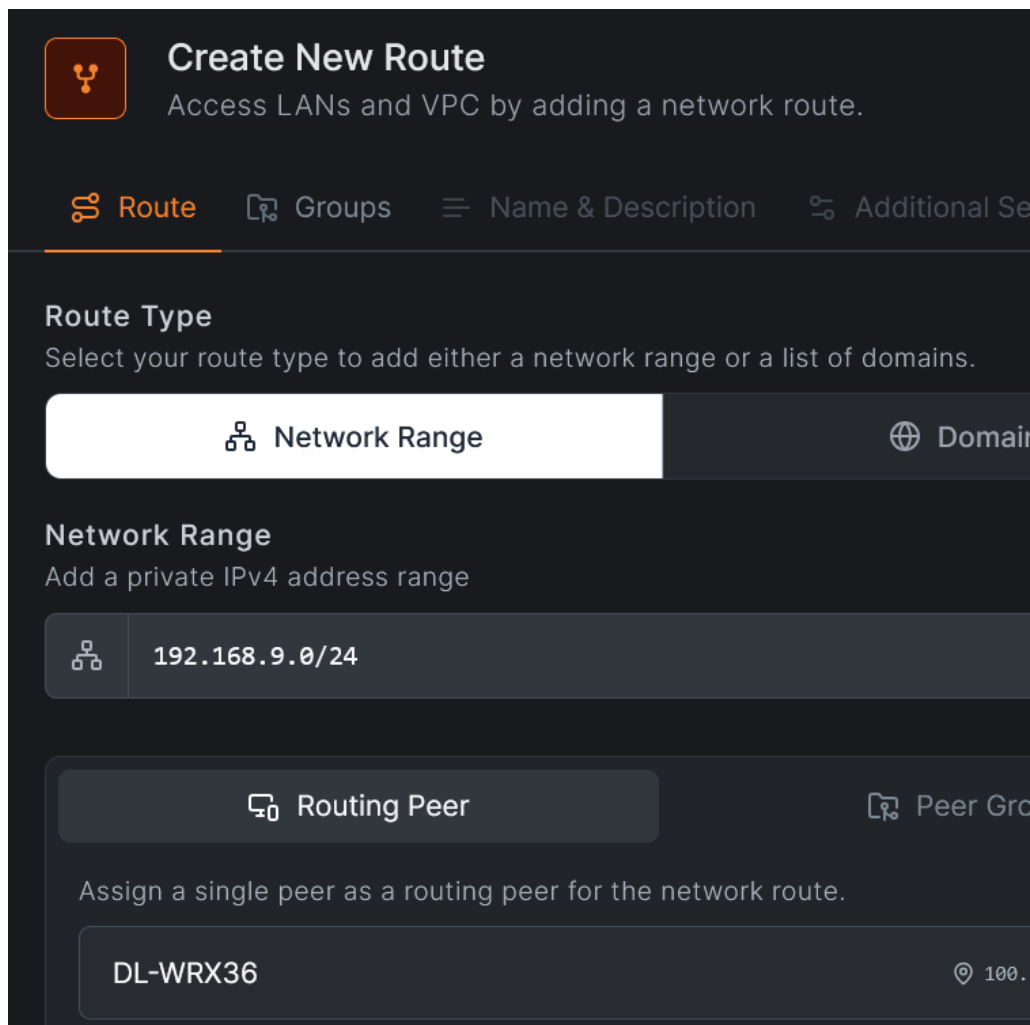
I will create a routing rule to create a route for this 192.168.9.0/24 subnet to my DL-WRX36 and push that route to all peers.

Those pushed routes are pushed to an alternate routing table on all peers, this table is usually called netbird.

Lets go:

NetBird Dashboard> Network Routes > Add Route

Add the network range to my DL-WRX36:



The screenshot shows the 'Create New Route' interface in the NetBird dashboard. At the top, there's a title 'Create New Route' with a subtitle 'Access LANs and VPC by adding a network route.' Below this is a navigation bar with tabs: 'Route' (selected), 'Groups', 'Name & Description', and 'Additional Settings'. The main section is titled 'Route Type' with the instruction 'Select your route type to add either a network range or a list of domains.' There are two buttons: 'Network Range' (selected) and 'Domain'. Under 'Network Range', there's a sub-instruction 'Add a private IPv4 address range' and a text input field containing '192.168.9.0/24'. Below this is another section titled 'Routing Peer' with the instruction 'Assign a single peer as a routing peer for the network route.' It features a button 'Routing Peer' and a text input field containing 'DL-WRX36'.

Advertise this route to all my peers:



Create New Route

Access LANs and VPC by adding a network route.

 Route  **Groups**  Name & Description  Additional Settings

Distribution Groups

Advertise this route to peers that belong to the following groups

 Routing Peers ×

Access Control Groups (optional)

These groups allow you to limit access to this route. Simply use these groups as a destination when creating access policies.

Add or select group(s)...

[Learn more about Network Routes](#) 

BackContinue

Name and description:



Create New Route

Access LANs and VPC by adding a network route.

 Route  Groups  **Name & Description**  Additional Settings

Network Identifier

Add a unique network identifier that is assigned to each device.

DL-WRX36

Description (optional)

Write a short description to add more context to this route.

Route to DL-WRX36 192.168.9.0/24 subnet|

Additional settings:

 **Create New Route**
Access LANs and VPC by adding a network route.

[Route](#) [Groups](#) [Name & Description](#) [Additional Settings](#)

Enable Route
Use this switch to enable or disable the route. ☒

Masquerade
Allow access to your private networks without configuring routes on your local routers or other devices. ☒

Metric
A lower metric indicates higher priority.

9999

You might need to restart NetBird on all peers

On my Oracle VPS I can now see the rules and the alternate routing table created by NetBird:

```
ubuntu@vps-egc:~$ ip rule show
0:    from all lookup local
105:  from all lookup main suppress_prefixlength 0
110:  not from all fwmark 0x1bd00 lookup netbird
32766: from all lookup main
32767: from all lookup default
ubuntu@vps-egc:~$
```

```
ubuntu@vps-egc:~$ ip route show table netbird
192.168.9.0/24 dev wt0
ubuntu@vps-egc:~$
```

wg show should also show the 192.168.9.0/24 added to the correct peer:

```
ubuntu@vps-egc:~$ sudo wg show
peer: < peer key >
  endpoint: [XXXXX:fedf]:33423
  allowed ips: 100.211.224.116/32, 192.168.9.0/24
  latest handshake: 42 seconds ago
  transfer: 1.41 KiB received, 2.00 KiB sent
  persistent keepalive: every 25 seconds

peer: < peer key >
  endpoint: [XXXX:1000:bea5:11ff:fe3e]:51555
  allowed ips: 192.168.5.0/24, 100.211.152.75/32
  latest handshake: 1 minute, 39 seconds ago
  transfer: 156 B received, 392 B sent
  persistent keepalive: every 25 seconds
```

So from my oracle VPS there now is a route to my DL-WRX36 subnet

Create Networks

See: <https://docs.netbird.io/how-to/networks>

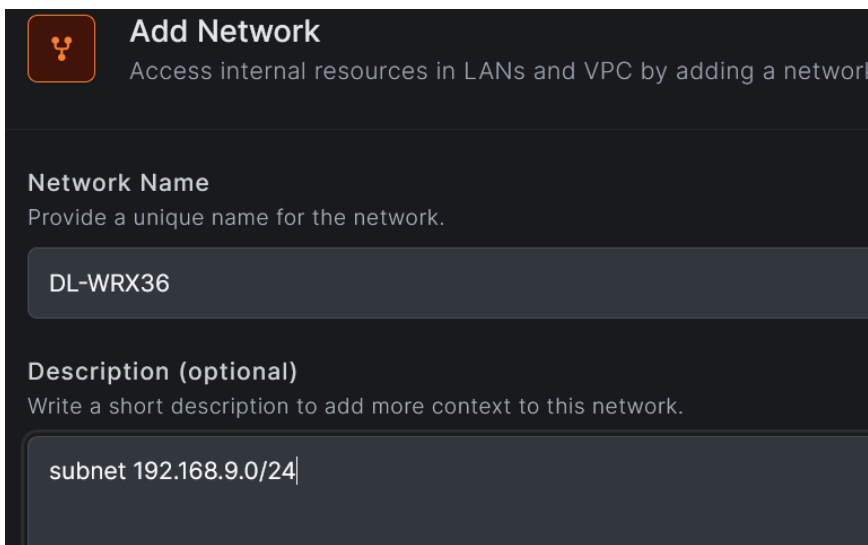
As I have just a few routers and a VPS to connect I use `Network Routes` which is simpler then using Networks.

Networks is new and has finer grained control but needs more work to setup and lacks support for exit nodes so that has still to be done with [Network Routes](#).

For OpenWRT the networks are often simple, so basically a Network has one routing peer which is the router or appliance in that network which holds the NetBird connection. This connects NetBird with the resources of the routing peer so basically the subnet or an IP address of a server which is running on this subnet. In this case I have my subnet as resource.

The access is controlled by Access Policies more on that later.

Start with creating a new Network e.g. the Network of my DL-WRX36 router which has subnet 192.168.9.0/24 and I want everyone to have access to this subnet:



Add Network
Access internal resources in LANs and VPC by adding a network

Network Name
Provide a unique name for the network.

DL-WRX36

Description (optional)
Write a short description to add more context to this network.

subnet 192.168.9.0/24

Proceed with making a **new Resource**:

Fill in name and address, under **Destination Groups** (these are the Access Control List > Groups) make a new group which later will hold our routing peer (added later to this the Access Control Destination group). Just enter the text for the new group e.g. DL-WRX36-group



Add Resource

Add new resource to "DL-WRX36"

Name

Provide a name for your resource

Description (optional)

Write a short description to add more context to this resource.

Address

Enter a single IP address, CIDR block or domain name

Destination Groups

Add this resource to groups and use them as destinations when creating policies





 DL-WRX36-group

Add this group by pressing **'Enter'**

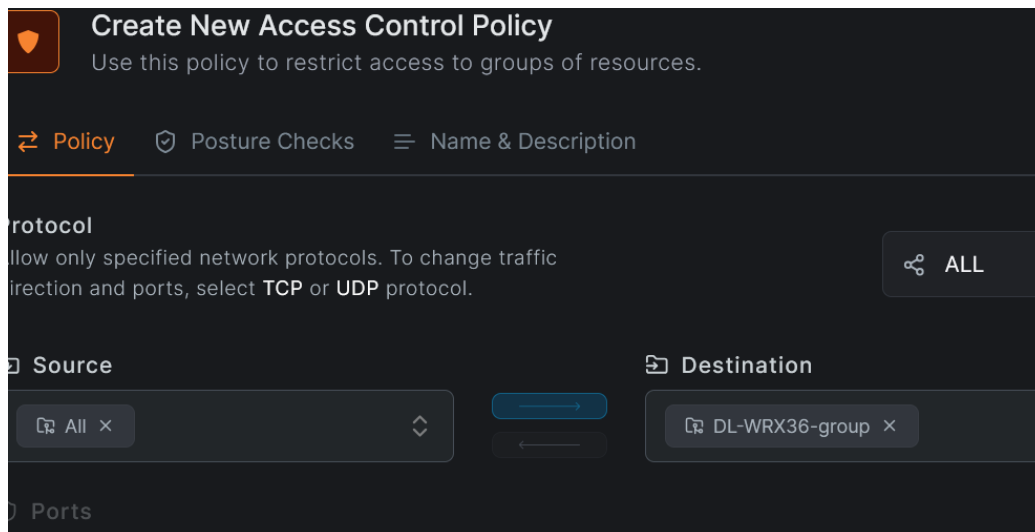
Proceed with making an Access Policy.

The destination is automatically your newly created Destination group (we will later add our peer to this Access group).

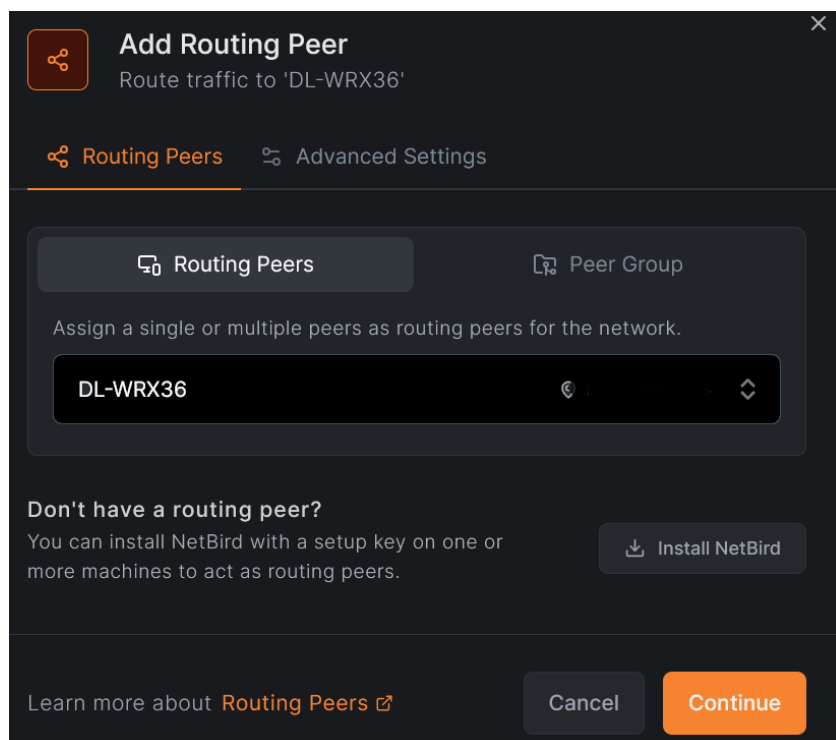
As Source I use `All` as everything in my network can have access but you can restrict it to your liking.

Note:

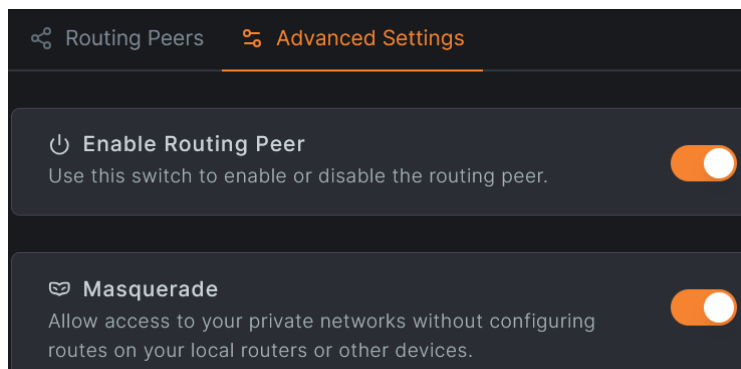
For every resource you have to make an Access Policy as the Destination Group you added is not automatically added to the `All` group



Proceed with adding a Routing Peer which is of course my DL-WRX36 peer:

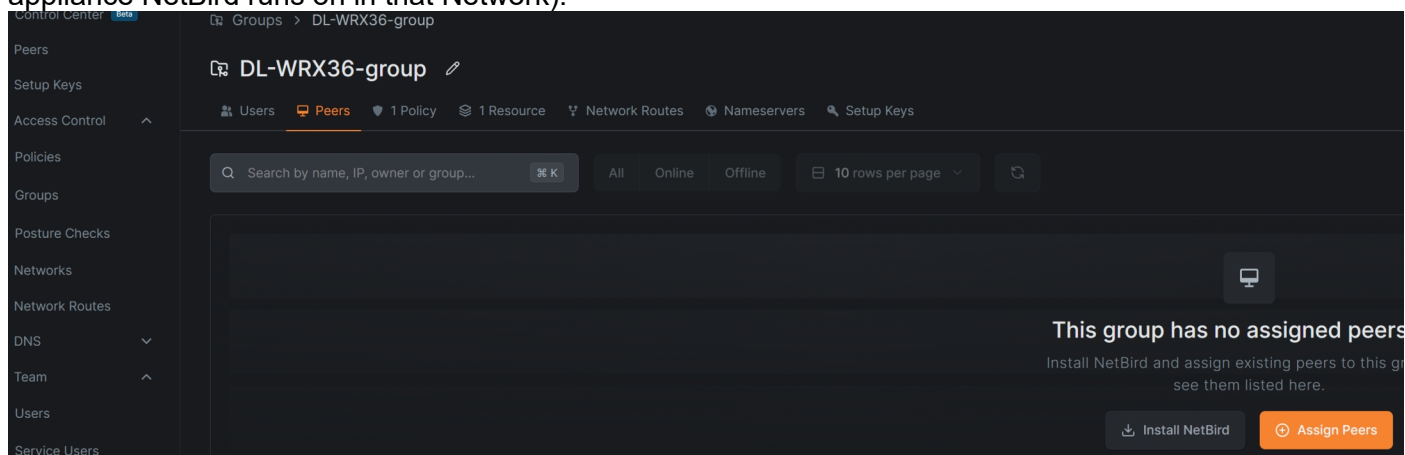


On Advanced settings enable Masquerading:



The last step is to add the DL-WRX36 peer to the Access Control Group DL-WRX36-group.

Goto Access Control > Groups and open the DL-WRX36 -group > Peers and assign you peer (the router or appliance NetBird runs on in that Network):



What I found confusing is that when making a resource you have to add the destination group. The Destination group is an Access List group to control access and it is only possible to control access by group and not by individual peer so you have to make a new Access control group and add your peer later to that group.

DNS settings

See:

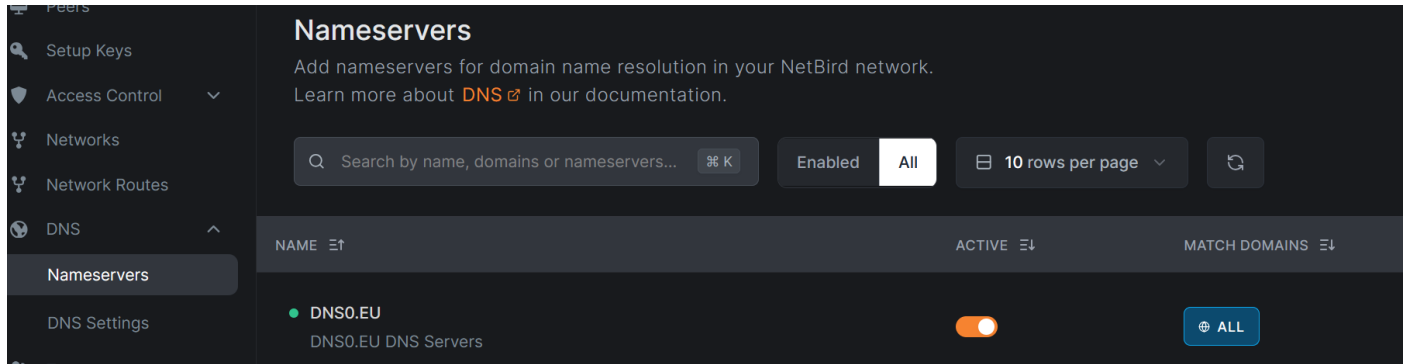
<https://docs.netbird.io/how-to/manage-dns-in-your-network>

<https://forum.openwrt.org/t/using-netbird-with-dnsmasq/218358/3?u=egc>

NetBird by default runs its own DNS server on the peers, this is included in the NetBird executable and is used by default.

So make sure you set a Nameserver under DNS settings:

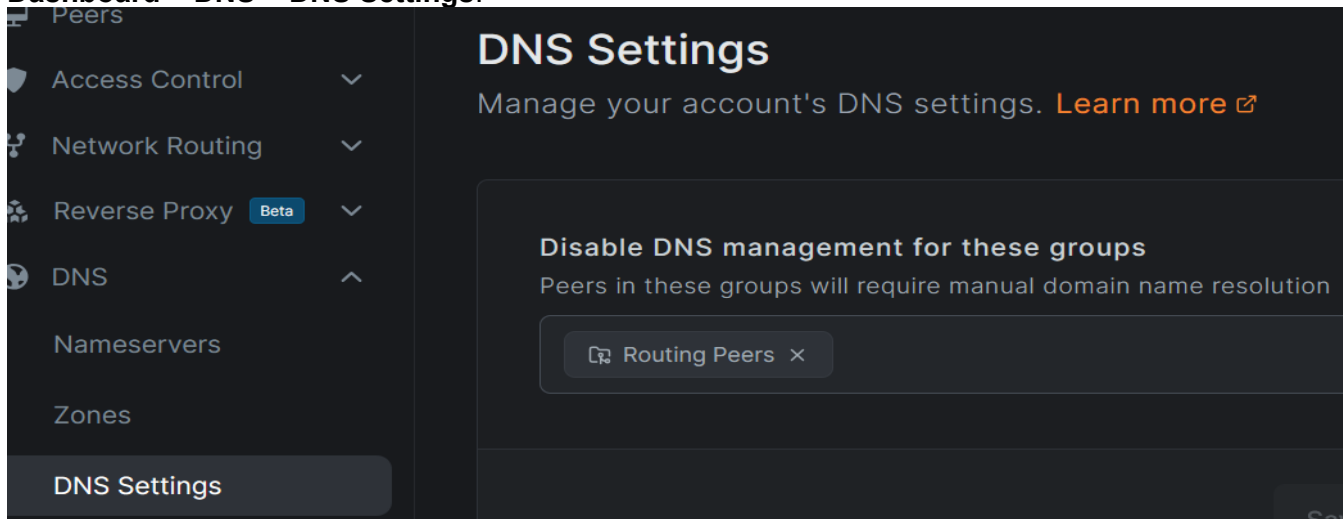
Dashboard > DNS > Nameservers:



I have chosen the DNS0.EU nameserver but you can choose others from a list or add your own.

If you do not want to use the NetBird DNS on your peers than you can disable it:

Dashboard > DNS > DNS Settings:



In this example I have disabled it my routing peers, I have it Enabled for my central VPS

If you have one central DNS server you can set the NetBird name (e.g. mywrx36.netbird.cloud) of that server as Nameserver

When DNS is enabled for the peer it will adapt `/etc/resolv.conf` (the file which the router itself uses for DNS resolution and which normally points to 127.0.0.1 where DNSMasq is listening)

It changes the nameserver to:

```
search netbird.cloud lan
nameserver 127.0.0.153
```

This is the address the netbird DNS server is listening on so with this active you do not have any local name resolution of the router. This is probably not what you want. You can disable Dns form the Dashnoard as shown or on the routier itself: `netbird down && netbird up --disable-dns=true`

If you want to have name resolution of Netbird nodes while you have DNS on the peers **disabled** you can add to DNSMasq `list server '/netbird.cloud/100.65.X.X` where 100.105.X.X is a central netbird node for which you have DNS enabled.

Make sure to place `netbird.cloud` on the Domain whitelist or Disable rebind protection!

Create Exit node

An exit node is a peer which acts as a VPN server other designated peers route all their traffic via the exit node.

On the exit node it is important that the firewall allows forwarding from **netbird** to **wan**, see paragraph about [firewall](#).

NetBird documentation: <https://netbird.io/knowledge-hub/netbird-network-routes>, scroll down to the bottom.

Log in in the NetBird dashboard

Peers > Click on the peer you want to be the exit node > On the overview page scroll to the bottom and click **Setup Exit node**

DL-WRX36 [edit icon]

Cancel Save Changes

NetBird IP Address 100... [edit icon] [copy icon]

Public IP Address 2001... [copy icon]

Domain Name dl-wrx36.netbird.cloud [copy icon]

Hostname DL-WRX36 [copy icon]

Region The Netherlands [copy icon]

Operating System OpenWrt snapshot

Registered on 8 October, 2025 at 6:53 PM (2 days ago)

Last seen just now

Agent Version 0.58.2

Session Expiration [toggle off]
Enable to require SSO login peers to re-authenticate when their session expires after a certain period of time.

SSH Access [toggle on]
Enable the SSH server on this peer to access the machine via an secure shell.

Remote Access
Connect directly to this peer via SSH or RDP.
[button: SSH]

Assigned Groups
Use groups to control what this peer can access.
[button: All] [button: Routing Peers x] [dropdown arrow]

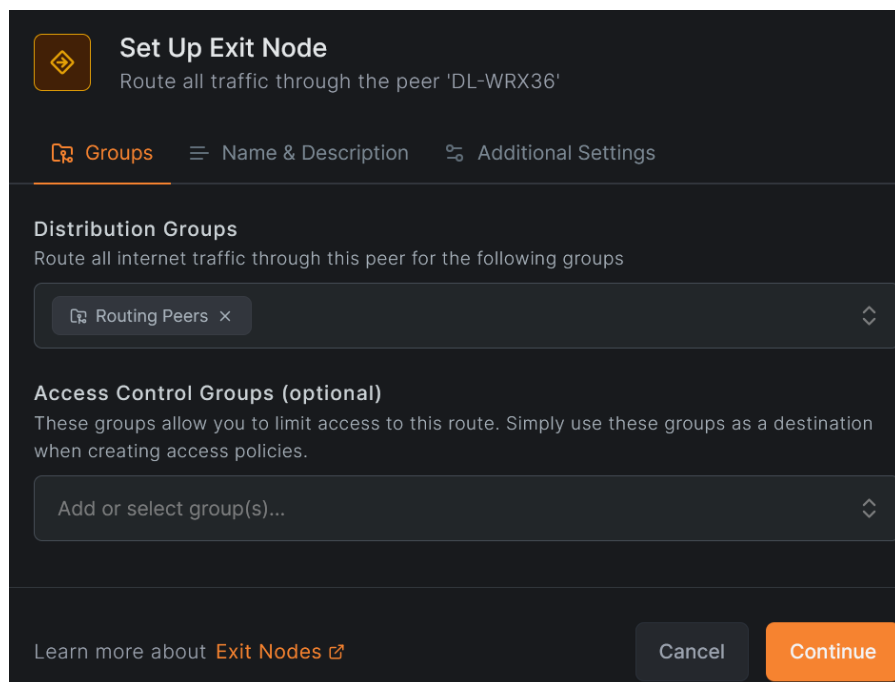
Network Routes [button: Accessible Peers] [button: Traffic Events]

Access other networks without installing NetBird on every resource.

[button: Set Up Exit Node] [button: Add Route v]

NAME	NETWORK	DISTRIBUTION GROUPS	ACTIVE	
DL-WRX36	192.168.9.0/24	[button: Routing Peers]	[toggle on]	[button: Delete]

Under **Groups** add the peers you want to use the exit node, I had created a group **Routing Peers** and I want all those peers to use this router as exit node



Set Up Exit Node
Route all traffic through the peer 'DL-WRX36'

Groups | Name & Description | Additional Settings

Distribution Groups
Route all internet traffic through this peer for the following groups

Routing Peers x

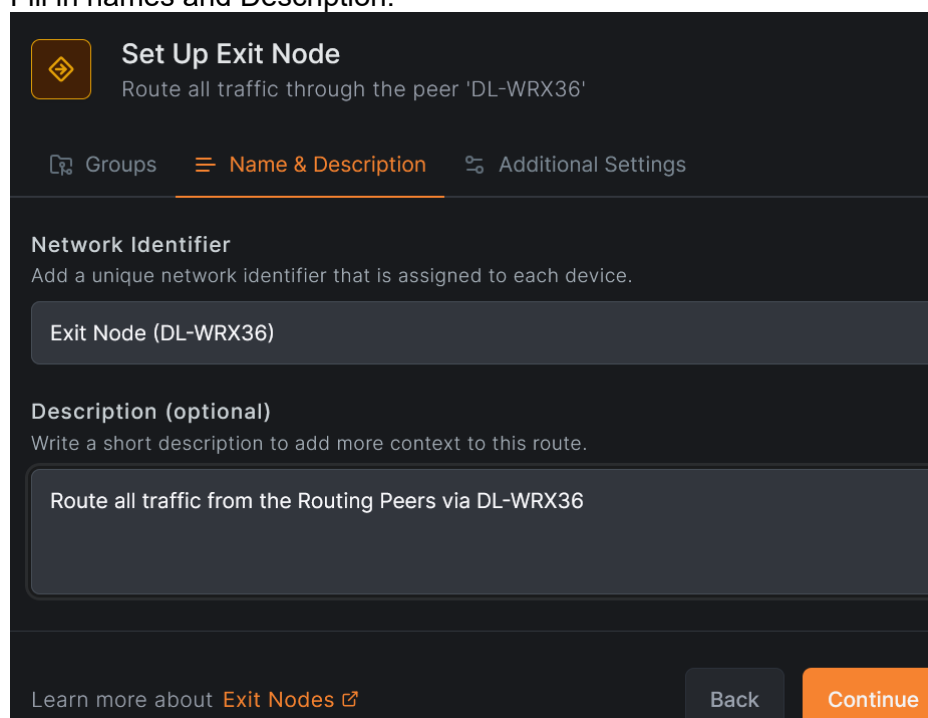
Access Control Groups (optional)
These groups allow you to limit access to this route. Simply use these groups as a destination when creating access policies.

Add or select group(s)...

Learn more about [Exit Nodes](#) | [Cancel](#) | [Continue](#)

Continue

Fill in names and Description:



Set Up Exit Node
Route all traffic through the peer 'DL-WRX36'

[Groups](#) | **Name & Description** | [Additional Settings](#)

Network Identifier
Add a unique network identifier that is assigned to each device.

Exit Node (DL-WRX36)

Description (optional)
Write a short description to add more context to this route.

Route all traffic from the Routing Peers via DL-WRX36

Learn more about [Exit Nodes](#) | [Back](#) | [Continue](#)

Continue

Enable Route and Auto Apply Route



Set Up Exit Node

Route all traffic through the peer 'DL-WRX36'

[Groups](#)
[Name & Description](#)
[Additional Settings](#)


Enable Route

Use this switch to enable or disable the route.


Auto Apply Route

Automatically apply this exit node to your distribution groups. This requires NetBird client v0.55.0 or higher.

Metric

A lower metric indicates higher priority routes.



9999



[Learn more about Exit Nodes](#)
[Back](#)
[+ Add Exit Node](#)

Add Exit Node

My DL-WRX36 is running Snapshot with NetBird 0.58 (you can see it on the overview page if you click on the Peer) so all routes are applied automatically.

My DL-WRX36 now has set a route to its own subnet (which is 192.168.9.0/24), pushed to all the Routing peers en an Exit node which pushes a default route to all the routing peers.

[Network Routes](#)
[Accessible Peers](#)
[Traffic Events](#)

2 Network Routes

Access other networks without installing NetBird on every resource.

NAME	NETWORK	DISTRIBUTION GROUPS	ACTIVE
Exit Node (DL-WRX36)	Exit Node	Routing Peers	
DL-WRX36	192.168.9.0/24	Routing Peers	

You can check on one of the other routing peers e.g. my R7800-2 where you can see the pushed default route and the pushed route to reach the DL-WRX36:

```
root@R7800-2:~# ip route show table netbird
default dev wt0
192.168.9.0/24 dev wt0
```

Now all traffic from the R7800-2 (and all its LAN clients are routed) via NetBird interface, which is actually your WireGuard interface and `wg show` will show you how the routing is taking place via the exit node

NetBird together with other VPN interfaces and Policy Based Routing

If you use other VPN connections together with Netbird it is recommended that your default route is via the WAN interface so that Netbird will use this WAN interface to connect to its peers and not the VPN interface, otherwise your Netbird connection might be slow or unstable.

For specific routing via Netbird or other VPN interfaces you can use Policy Based Routing. Policy based routing can be used to route traffic via different routes according to source and/or destination. It can be useful if you have set your peer (router) to use an exit node which means all traffic is routed via the specified exit node and you want to route some traffic via the WAN instead of the exit node.

You can setup this up [manually](#) but it is often easier to install the [PBR app](#). When using the [PBR app](#) you have to take the following points into account:

IP rule priority

To let PBR do its work the PBR ip rules have to come before the Netbird ip rules. The Netbird ip rules start normally just above 100 so let the PBR ip rules start at 99 (it counts downward) instead of the default 30000.
(Note this will limit the maximum number of interface/devices you can use to 98 instead of 256)

/etc/config/pbr:

```
option uplink_ip_rules_priority '99'
```

or from command line:

```
uci set pbr.config.uplink_ip_rules_priority="99"
uci commit pbr
service pbr restart
```

This will make sure the PBR ip rules will come before the NetBird rules (>100).

If everything is up you should see something like this:

```
root@R7800-2:~# ip rule show
0:      from all lookup local
97:      from all sport 52199 lookup pbr_wan
97:      from all lookup main suppress_prefixlength 1
98:      from all fwmark 0x20000/0xff0000 lookup pbr_netbird1
99:      from all fwmark 0x10000/0xff0000 lookup pbr_wan
105:     from all lookup main suppress_prefixlength 0
110:     not from all fwmark 0x1bd00 lookup netbird
32766:   from all lookup main
32767:   from all lookup default
```

Note the PBR rules **before** the Netbird rules, so these take precedence.

Netbird starts rather late so you might need to restart PBR (*service pbr restart*) after setting up or after a reboot

Overlapping FWMARK

Netbird is using an fwmark of 0x1bd00 for the WG interface but that is not the only fwmark it uses. It uses more fwmarks in that same range in the firewall to route to management and signal plane. (You can check the principal fwmark of Netbird with *wg show*, it is listed under the interface section of the netbird interface)

These overlap with the standard PBR fwmark and fwmask. To resolve this change fwmark from default 00010000 and fwmask from default 00ff0000 to

```
option uplink_mark '00100000'
option fw_mask '0ff00000'
```

Default route of Netbirds WG connection

Earlier it was recommended to have your default route via the WAN, so that netbird uses the WAN for its outward connection rather than a VPN — otherwise you end up with a tunnel inside a tunnel.

Running a tunnel in a tunnel is valid, and if you have no problems you can leave it, but you can run into MTU and performance problems — also because usually only a relayed connection is available and no peer-to-peer (P2P) connection. Routing via the WAN is preferable.

Note that pbr policies cannot target Netbird traffic at all: every pbr chain opens with `meta mark & 0xff0000 != 0 return`, so a policy naming a Netbird endpoint installs cleanly but will sit at zero counters. Use the ip rule below instead.

To check which route Netbird's WireGuard traffic actually takes, resolve the route to a peer's endpoint IP using Netbird's fwmark. Get the endpoints with `wg show` (take the IP address without its port):

```
ip route get <endpoint ip of a peer in wt0> mark 0x1bd00
```

Netbird's fwmark itself can be read with:

```
wg show all fwmark
```

If that reports `off`, Netbird is not in fwmark mode and this section does not apply.

You can use that fwmark to add an ip rule routing the traffic via the `pbr_wan` table. Give the rule the same priority number as pbr's `from all lookup main suppress_prefixlength 1` rule — it will then show up directly below that rule in `ip rule show`, which is where it needs to be to take effect before pbr's own rules:

```
ip rule add fwmark 0x1bd00 lookup pbr_wan priority <priority>
```

Since that priority is inside the range pbr clears on stop/reload, add it as a pbr include so it is reapplied automatically:

```
#!/bin/sh
# get prefixlength from pbr config (default is 1)
pfl="$(uci -q get pbr.config.prefixlength)"
pfl="${pfl:-1}"

# netbird's fwmark, viewable with 'wg show all fwmark' or under the interface
# section of 'wg show' for the netbird interface (wt0)
fwmark="0x1bd00"

# priority of pbr's 'lookup main suppress_prefixlength' rule
prio="$(ip rule show | awk -v pfl="$pfl" \
  '$0 ~ "lookup main suppress_prefixlength " pfl "[ \t]$" {sub(/:/, "", $1); print $1; exit}')"

# nothing to do if pbr has not installed its rules (yet)
[ -n "$prio" ] || exit 0

ip rule del fwmark "$fwmark" lookup pbr_wan priority "$prio" 2>/dev/null
ip rule add fwmark "$fwmark" lookup pbr_wan priority "$prio"
```

You can add this script as include script in PBR:

```
cp yourscript.sh /usr/share/pbr/pbr.user.netbird
```

then add to `/etc/config/pbr`:

```
config include
```

```
option path '/usr/share/pbr/pbr.user.netbird'
option enabled '1'
```

Verify with ``ip route get <endpoint ip> mark 0x1bd00`` — it should now report the WAN device.

This covers IPv4 only (``ip rule`` without ``-6``). If a netbird peer endpoint is reached over IPv6, mirror the rule with ``ip -6 rule`` and set ``ipv6_enabled '1'`` in the pbr config.

For checking rules and routes see the Check and Troubleshoot section

IPv6 and Netbird

For IPv6 make sure you do not have `8000::/1` and `0::/1` in the Allowed IPs but just `0::/0` and make sure sourcerouting is disabled on the wan6 interface:

```
config interface 'wan6'
    option device 'wan'
    option proto 'dhcipv6'
    option reqaddress 'try'
    option reqprefix '62'
    option norelease '1'
    option sourcefilter '0'    <<<<<<<THIS
```

IPv6

Starting with version 0.71 using IPv6 is possible, see:

<https://docs.netbird.io/manage/settings/ipv6>

Support

For support and questions see the NetBird support thread:

<https://forum.openwrt.org/t/netbird-support-discussion-thread/237831/8>

Known Problems

SSH-Access from Dashboard

Enabling [Lazy Connections](#) might stop SSH access from the Dashboard (Settings > Clients):

<https://netbird.io/knowledge-hub/lazy-connections>

<https://forum.openwrt.org/t/netbird-support-discussion-thread/237831/44?u=egc>

Install on Oracle VPS with Ubuntu (24.04)

```
sudo apt-get update
sudo apt install ca-certificates curl gnupg -y
curl -sSL https://pkgs.netbird.io/debian/public.key | sudo gpg --dearmor --output /usr/share/keyrings/netbird-
archive-keyring.gpg
echo 'deb [signed-by=/usr/share/keyrings/netbird-archive-keyring.gpg] https://pkgs.netbird.io/debian stable
main' | sudo tee /etc/apt/sources.list.d/netbird.list
```

```
sudo apt-get update
sudo apt-get install netbird
# only for the GUI
#sudo apt-get install netbird-ui
```

```
netbird up --setup-key <setup-key made on dashboard> --allow-server-ssh
```

Log on Ubuntu: `cat /var/log/netbird/client.log`

SSH access note that the user name is usually: *ubuntu*

For (SSH) Access add the firewall rules

```
sudo iptables -I INPUT 3 -p udp --dport 3478 -j ACCEPT # NetBird TURN
sudo iptables -I INPUT 4 -p tcp --dport 44338 -j ACCEPT # SSH service port from NetBird

sudo iptables -I INPUT 5 -p udp --dport 51820 -j ACCEPT # NetBird WireGuard
#sudo iptables -t nat -I POSTROUTING -o wt0 -j MASQUERADE #To Masquerade traffic ?
#sudo iptables -t nat -I POSTROUTING -o ens3 -j MASQUERADE #To Masquerade traffic ?
```

Make persistent:
`sudo netfilter-persistent save`

vcn-XXX > Security > Default Security List for vcn-XXX > Security rules:

☐	No	0.0.0.0/0	UDP	All	3478
☐	No	0.0.0.0/0	TCP	All	44338
☐	No	0.0.0.0/0	UDP	All	51820

Throughput improvements via transport layer offloading

Tuning two features may show improved throughput:

- `rx-udp-gro-forwarding`: Enables UDP Generic Receive Offload (GRO) forwarding, which aggregates incoming UDP packets to reduce CPU overhead on receive.
- `rx-gro-list`: If disabled (off), it prevents multiple flows from being aggregated simultaneously which simplifies flow handling and performance on some workloads.

From command line:

1. Install "ethtool":

```
apk update
apk add ethtool
```

2. Apply the changes:

Substitute "wan" below for your WAN interface.</WRAP>

```
ethtool -K wan rx-gro-list off
ethtool -K wan rx-udp-gro-forwarding on
```

3. Test the changes before and after before committing them permanently with something similar to the following commands.

You want to verify:

- Packet aggregation is working as measured by reduced packets/sec on the wire with GRO enabled (verify with tools like: `ethtool -S <interface> | grep udp` or `netstat -su`)
- CPU usage is reduced. Lower CPU usage on the receiver compared to same test with `rx-udp-gro-forwarding` turned off
- High throughput is achieved near line rate (e.g., 1 Gbps, 10Gbps, etc) without packetloss. You need `iperf3` for proper measurement

Setup Oracle free OpenVPN cloud server

<https://www.youtube.com/watch?v=E-CLtExRzX8>

<https://mateo.cogeanu.com/2020/wireguard-vpn-pihole-on-free-oracle-cloud/>

References

NetBird wiki: <https://openwrt.org/docs/guide-user/services/vpn/netbird>)

NetBird support thread: <https://forum.openwrt.org/t/netbird-support-discussion-thread/237831/8>

Upgrade from 0.50 to 0.58: <https://github.com/netbirdio/netbird/issues/4322>

NetBird Releases

<https://github.com/netbirdio/netbird/releases>

Configuration

See: <https://docs.netbird.io/get-started/cli>

Flags are for using on the command line e.g. **netbird up --flag** if these are also config items then it will be stored in the config in `/root/.config/netbird/default.json` so there is no need to add the flag again. Most flags can also be set as Environment variable with syntax `NB_FLAG` and using underscores instead of hyphens e.g.: `export NB_FLAG=1` or set in the environment in the init script: `procd_append_param env NB_DISABLE_SSH_CONFIG="1"`

Disable SSH Authorization per user

flag (command line): `--disable-ssh-auth`
env param: `NB_DISABLE_SSH_AUTH=1`
config: `"DisableSSHAUTH": true,`

Enable SSH root user

flag: `--enable-ssh-root`
env param: `NB_ENABLE_SSH_ROOT`
config: `EnableSSHRoot": true,`

Disable Netbird DNS, this disables the writing to `resolv.conf`

config: `"DisableDNS": true,`

Disable SSH integration with OpenSSH, this disables the writing of `/etc/ssh/ssh_config.d/99-netbird.conf`:

flag: `- --disable-ssh-config`
env param: `NB_DISABLE_SSH_CONFIG=1`
config: `"DisableSSHConfig": true`

Addendum 1

/root/.config/netbird/defaulttjson:

```
{
  "PrivateKey": "kDJR2ykm0=",
  "PreSharedKey": "",
  "ManagementURL": {
    "Scheme": "https",
    "Opaque": "",
    "User": null,
    "Host": "api.netbird.io:443",
    "Path": "",
    "RawPath": "",
    "OmitHost": false,
    "ForceQuery": false,
    "RawQuery": "",
    "Fragment": "",
    "RawFragment": ""
  },
  "AdminURL": {
    "Scheme": "https",
    "Opaque": "",
    "User": null,
    "Host": "app.netbird.io:443",
    "Path": "",
    "RawPath": "",
    "OmitHost": false,
    "ForceQuery": false,
    "RawQuery": "",
    "Fragment": "",
    "RawFragment": ""
  },
  "WgIface": "wt0",
  "WgPort": 51820,
  "NetworkMonitor": null,
  "IFaceBlackList": [
    "wt0",
    "wt",
    "utun",
    "tun0",
    "zt",
    "ZeroTier",
    "wg",
    "ts",
    "Tailscale",
    "tailscale",
    "docker",
    "veth",
    "br-",
    "lo"
  ],
  "DisableIPv6Discovery": false,
  "RosenpassEnabled": false,
  "RosenpassPermissive": false,
  "ServerSSHAllowed": true,
  "EnableSSHRoot": true,
  "EnableSSHSFTP": null,
  "EnableSSHLocalPortForwarding": null,
  "EnableSSHRemotePortForwarding": null,
}
```

```
"DisableSSHAAuth": true,  
"SSHJWTCacheTTL": null,  
"DisableClientRoutes": false,  
"DisableServerRoutes": false,  
"DisableDNS": true,  
"DisableFirewall": false,  
"BlockLANAccess": false,  
"BlockInbound": false,  
"DisableNotifications": true,  
"DNSLabels": null,  
"SSHKey": "-----BEGIN PRIVATE KEY-----\nMC4CA\n-----END PRIVATE KEY-----\n",  
"NATExternalIPs": null,  
"CustomDNSAddress": "",  
"DisableAutoConnect": false,  
"DNSRouteInterval": 60000000000,  
"ClientCertPath": "",  
"ClientCertKeyPath": "",  
"LazyConnectionEnabled": false,  
"MTU": 1280  
}
```

```
GO_VERSION_MAJOR_MINOR:=1.25
GO_VERSION_PATCH:=7
GO_VERSION_RC:=
GO_BOOTSTRAP_VERSION:=bootstrap
#PKG_HASH:=c9132a8a1f6bd2aa4aad1d74b8231d95274950483a4950657ee6c56e6e817790
PKG_HASH:=178f2832820274b43e177d32f06a3ebb0129e427dd20a5e4c88df2c1763cf10a
```


-
- -
 -
 -
 -
-

-
- -
-

-
-

-
-
-

- -
 -
-

- -
 -
 -
 -
-

- -
-

- 1.
- 2.

Variable	Equivalent flag	Description
NB_ADMIN_URL	--admin-url	Admin Panel URL (default <code>https://app.netbird.io:443</code>)
NB_ANONYMIZE	-A, --anonymize	Anonymize IPs and non-netbird.io domains in logs/status
NB_CONFIG	-c, --config	Path to profile config file (deprecated on up/login)
NB_DAEMON_ADDR	--daemon-addr	Daemon service address (default <code>unix:///var/run/netbird.sock</code>)
NB_HOSTNAME	-n, --hostname	Custom device hostname
NB_LOG_FILE	--log-file	Log output path(s): file path, console, or syslog. Comma-separated for multiple (default <code>/var/log/netbird/client.log</code>)
NB_LOG_LEVEL	-l, --log-level	Log level (default info)
NB_MANAGEMENT_URL	-m, --management-url	Management Service URL (default <code>https://api.netbird.io:443</code>)
NB_PRESHARED_KEY	--preshared-key	WireGuard PreSharedKey
NB_SERVICE	-s, --service	System service name (default netbird)
NB_SETUP_KEY	-k, --setup-key	Setup key to register peer
NB_SETUP_KEY_FILE	--setup-key-file	Path to file containing setup key

Variable	Equivalent flag	Description
NB_ALLOW_SERVER_SSH	--allow-server-ssh	Allow SSH server on peer
NB_BLOCK_INBOUND	--block-inbound	Block all inbound connections, overrides policies
NB_BLOCK_LAN_ACCESS	--block-lan-access	Block LAN access when acting as peer

Variable	Equivalent flag	Description
NB_DISABLE_AUTO_CONNECT	--disable-auto-connect	router/exit node Don't auto-connect when service starts
NB_DISABLE_CLIENT_ROUTES	--disable-client-routes	Don't process client routes from management
NB_DISABLE_DNS	--disable-dns	Don't configure DNS settings
NB_DISABLE_FIREWALL	--disable-firewall	Don't modify firewall rules
NB_DISABLE_IPV6	--disable-ipv6	Disable IPv6 overlay address (v0.68.0+)
NB_DISABLE_SERVER_ROUTES	--disable-server-routes	Don't act as route for server routes
NB_DISABLE_SSH_AUTH	--disable-ssh-auth	Disable SSH JMW authentication
NB_DNS_RESOLVER_ADDRESS	--dns-resolver-address	Custom local DNS resolver address e.g. 127.0.0.1:50
NB_DNS_ROUTER_INTERVAL	--dns-router-interval	DNS route update interval (default 1m0s)
NB_ENABLE_LAZY_CONNECTION	--enable-lazy-connection	[Experimental] Establish connections on- demand
NB_ENABLE_ROSENPASS	--enable-rosenpass	[Experimental] Post-quantum secured connection
NB_ENABLE_SSH_LOCAL_PORT_FORWARDING	--enable-ssh-local-port-forwarding	Enable local SS port forwarding
NB_ENABLE_SSH_REMOTE_PORT_FORWARDING	--enable-ssh-remote-port-forwarding	Enable remote SSH port forwarding
NB_ENABLE_SSH_ROOT	--enable-ssh-root	Enable root login for SSH server
NB_ENABLE_SSH_SFTP	--enable-ssh-sftp	Enable SFTP subsystem for SSH server
NB_EXTERNAL_IP_MAP	--external-ip-map	Map external IP to local addresses/interfaces
NB_EXTRA_DNS_LABELS	--extra-dns-labels	Up to 32 comma separated DNS labels
NB_EXTRA_IFACE_BLACKLIST	--extra-iface-blacklist	Extra interfaces ignore for listeni
NB_FOREGROUND_MODE	-F, --foreground-mode	Start service in foreground

Variable	Equivalent flag	Description
NB_INTERFACE_NAME	--interface-name	WireGuard interface name (default wt0)
NB_MTU	--mtu	MTU for WireGuard interface (default 1280)
NB_NETWORK_MONITOR	-N, --network-monitor	Manage network monitoring (default: true on Win/macOS, false on Linux/FreeBSD)
NB_NO_BROWSER	--no-browser	Don't open browser for SSH login
NB_PROFILE	--profile	Profile name to use for login
NB_ROSENPASS_PERMISSIVE	--rosenpass-permissive	[Experimental] Allow non-Rosenpass peers
NB_SSH_JWT_CACHE_TTL	--ssh-jwt-cache-ttl	SSH JWT cache TTL in seconds (default 0, disabled; max useful ~600)
NB_WIREGUARD_PORT	--wireguard-port	WireGuard listening port (default 51820)

Variable	Equivalent flag	Description
NB_DISABLE_PROFILES	--disable-profiles	Lock client to the profile active at install time
NB_DISABLE_UPDATE_SETTINGS	--disable-update-settings	Prevent users from changing daemon settings via CLI/UI

Variable	Platform	Description
NB_USE_LEGACY_ROUTING	Linux	Bypass fwmark/ip-rule routing, use pre-0.26 method. Useful for exit-node connectivity issues (rp_filter, missing sysctls)
NB_SKIP_SOCKET_MARK	Linux	Alias for NB_USE_LEGACY_ROUTING
NB_DISABLE_CUSTOM_ROUTING	All	Revert to pre-exit-node routing behavior; rejects routes $\geq /7$ (e.g. 0.0.0.0/0)
NB_ROUTE_PROTO_FLAG	macOS, BSD	Custom route flag: 2 (RTF_PROTO2) or 3 (RTF_PROTO3); default RTF_PROTO1
NB_DISABLE_ROUTE_CACHE	Windows	Disable the 2-second cache on routing table lookups (debugging only)

Variable	Platform	Description
NB_WG_KERNEL_DISABLED	Linux	Force userspace WireGuard (wireguard-go) instead of the kernel module
NB_WG_DEBUG	All	Verbose WireGuard protocol logging (handshakes, keypairs, timers)
NB_DISABLE_EBPF_WG_PROXY	Linux	Disable eBPF WireGuard UDP proxy, fall back to classic UDP port proxy
NB_USE_NETSTACK_MODE	All	Run over gVisor netstack instead of a TUN device (needed in unprivileged containers)
NB_NETSTACK_SKIP_PROXY	All	In netstack mode, don't start the built-in SOCKS5 proxy
NB_SOCKS5_LISTENER_PORT	All	Netstack SOCKS5 proxy port (default 1080)

Variable	Platform	Description
NB_SKIP_NFTABLES_CHECK	Linux	Skip nftables probe at startup, use iptables directly
NB_NFTABLES_TABLE	Linux	Custom nftables table name (default netbird)
NB_DISABLE_CONNTRACK	All	Disable stateful connection tracking in the userspace filter
NB_DISABLE_USERSPACE_ROUTING	All	Prevent userspace filter from forwarding packets between interfaces
NB_DISABLE_MSS_CLAMPING	All	Stop rewriting TCP MSS to fit WireGuard MTU
NB_FORCE_USERSPACE_ROUTER	All	Force userspace packet forwarding even when native nftables/iptables is available
NB_ENABLE_LOCAL_FORWARDING	All	Allow userspace filter to forward to local (non-routed) addresses
NB_ENABLE_NETSTACK_LOCAL_FORWARDING	All	Same as above, but for netstack mode specifically
NB_USPFILTER_LOG_BUFFER	All	Queue size for userspace filter log messages (default 1000)

Variable	Platform	Description
NB_FORCE_RELAY	All	Skip P2P entirely; relay-only
NB_ICE_KEEP_ALIVE_INTERVAL_SEC	All	ICE keepalive interval, seconds (default 4)
NB_ICE_DISCONNECTED_TIMEOUT_SEC	All	Silence before marking disconnected, seconds (default 6)
NB_ICE_FAILED_TIMEOUT_SEC	All	Silence before falling back to relay, seconds (default 6)
NB_ICE_RELAY_ACCEPTANCE_MIN_WAIT_SEC	All	Min wait for a direct candidate before accepting relay (default 2)
NB_ICE_MONITOR_PERIOD	All	Interval between ICE health checks (default 5m)
NB_ENABLE_EXPERIMENTAL_LAZY_CONN	All	Connect to peers on-demand only (same as --enable-lazy-connection)
NB_LAZY_CONN_INACTIVITY_THRESHOLD	All	Idle time before tearing down a lazy connection (default 15m)

Variable	Platform	Description
NB_DNS_FORWARDER_PORT	All	Internal DNS forwarder port for DNS routes (default 22054)
NB_SKIP_DNS_PROBE	All	Skip startup reachability probe of the local resolver
NB_UNCLEAN_SHUTDOWN_RESOLV_FILE	Linux, FreeBSD	Backup path for /etc/resolv.conf, restored after unclean shutdown. Default <state-dir>/resolv.conf (/var/lib/netbird/resolv.conf on Linux, /var/db/netbird/resolv.conf on FreeBSD)

Variable	Platform	Description
NB_CONN_RETRY_INTERVAL_TIME	All	Initial wait before first reconnect attempt (e.g. 5s)
NB_CONN_MAX_RETRY_INTERVAL_TIME	All	Max backoff interval
NB_CONN_MAX_RETRY_TIME_TIME	All	Total retry time before giving up (double _TIME is intentional)
NB_CONN_RETRY_MULTIPLIER	All	Backoff growth factor (e.g. 1.7)

Variable	Platform	Description
NB_DISABLE_SSH_CONFIG	All	Don't write peer entries into system SSH config directory
NB_FORCE_SSH_CONFIG	All	Write SSH config even past the 200-peer default limit

Variable	Platform	Description
NB_RELAY_HC_ATTEMPT_THRESHOLD	All	Failed pings before switching relay servers (default 1)

Variable	Platform	Description
NB_MANAGEMENT_GRPC_MAX_MSG_SIZE	All	Max gRPC message size in bytes from management (raise past default 4MB)

Variable	Platform	Description
NB_LOG_FORMAT	All	json or syslog output format; default is human-readable text
NB_LOG_MAX_SIZE_MB	All	Max size per log file before built-in rotation (default 15)
NB_LOG_DISABLE_ROTATION	All	Disable built-in log rotation (e.g. to defer to system logrotate)

Variable	Platform	Description
NB_STATE_DIR	All	Directory for persistent state (config, state.json, WG keys). Default /var/lib/netbird (Linux/macOS), /var/db/netbird (FreeBSD), %PROGRAMDATA%\Netbird (Windows)
NB_DNS_STATE_FILE	All	Path to state.json (not DNS-only despite the name — covers DNS, SSH config, routes, auto-update state)
NB_ROSENPASS_LOG_LEVEL	All	Log level for the Rosenpass subprocess (default info)
NB_AUTO_UPDATE_DRY_RUN	All	Run auto-update check without replacing the binary
NB_PPROF_ADDR	All	Enable Go pprof HTTP endpoint, e.g. localhost:6060

Variable	Platform	Description
NB_METRICS_PUSH_ENABLED	All	Enable periodic telemetry push (default off)
NB_METRICS_INTERVAL	All	Metrics push interval (e.g. 60s)
NB_METRICS_SERVER_URL	All	Override metrics collection server URL
NB_METRICS_CONFIG_URL	All	Override remote metrics config URL

-
-
-
-

```
#!/bin/sh /etc/rc.common
```